

Chapter 9: Dependency Injection

1. Overview

Dependency Injection (DI) is the design pattern Angular is built around. Instead of a class constructing its own collaborators (`new HttpClient()`, `new Logger()`), it declares what it needs in its constructor (or via `inject()`), and a runtime container — the **injector** — hands it fully-configured instances. The class never knows *how* the dependency was built, only *what* it is.

Angular's DI system is not a bolt-on convenience; it is the substrate that makes services, testing, lazy loading, and hierarchical component trees work. Angular maintains a **tree of injectors** that mirrors two parallel structures at once:

- The **environment injector hierarchy** (module/standalone-based): `Platform` → `Root` → `Module/Route injectors`.
- The **element injector hierarchy** (component/directive-based): mirrors the DOM/component tree, one injector per component or directive that has providers.

When a class asks for a dependency, Angular walks up these trees looking for a matching **provider**.

Understanding this walk-up algorithm — and knowing exactly which injector wins when there's a conflict — is the single most interview-tested DI skill.

This chapter covers every provider recipe, every resolution modifier, the modern `inject()` function, and the internal mechanics of how instances get created, cached, and torn down.

2. Core Concepts

2.1 What Is an Injector?

An injector is an object with a `get(token)` method. Internally it holds a **provider record map** (token → recipe) and a **instance cache** (token → resolved singleton, per that injector). When asked for a token:

1. Check its own cache — if already instantiated, return the cached instance.
2. Check its own provider records — if a recipe exists, execute it, cache the result, return it.
3. Otherwise, delegate to its **parent injector** (unless a resolution modifier says not to).
4. If it's the root of the chain and nobody had it, delegate to the `NullInjector`, which throws `NullInjectorError: No provider for X`.

2.2 Two Injector Hierarchies

Since Angular 14+ (Ivy architecture, standalone-ready), there are officially **two distinct injector hierarchies**:

Hierarchy	Also called	Nodes	Lifetime
Environment injector hierarchy	"Module injector" historically	Platform injector → Root environment injector → Route-level environment injectors (lazy-loaded routes / <code>providers</code> on a <code>Route</code>)	Long-lived, app/route-scoped
Element injector hierarchy	"Node injector"	One per component/directive instance that declares <code>providers</code> / <code>viewProviders</code>	Tied to the component instance's lifecycle

The **environment injector** hierarchy is what `providedIn: 'root'`, `providedIn: 'platform'`, `NgModule.providers`, and standalone `bootstrapApplication({ providers })` populate. The **element injector** hierarchy is what `@Component({ providers })`, `@Component({ viewProviders })`, and `@Directive({ providers })` populate.

Resolution always starts at the **element injector of the requesting component** and walks up through element injectors (parent components) **before** falling through to the environment injector chain at the top. This is critical: a component-level provider always wins over a root-level provider, no matter how "singleton" the root service looks.

2.3 Provide Scopes

`providedIn: 'root'`

```
@Injectable({ providedIn: 'root' })
export class LoggerService {}
```

- Registers the service with the **root environment injector**, lazily — the provider record exists from app start, but the *instance* is only created the first time something injects it (tree-shakable).
- If the service is never injected anywhere, it's never instantiated, and with Ivy's tree-shaking, unused `providedIn: 'root'` services can even be removed from the bundle if nothing references the class.
- One instance per application (unless overridden lower in the tree).

`providedIn: 'platform'`

- Registers with the **platform injector**, which sits *above* the root injector and is shared across multiple Angular applications running on the same page (rare — used by things like `PlatformRef`, multi-app micro-frontend setups).
- One instance shared across all apps bootstrapped in that browser tab/process.

`providedIn: 'any'`

- A special scope: if the service is requested from an eagerly-loaded part of the app, it behaves like root (one shared instance). If requested from a **lazy-loaded module's** injector, that lazy module gets **its own separate instance**. This existed primarily for the `NgModule`-based lazy loading world.

Module-level (`NgModule.providers`)

```
@NgModule({ providers: [LoggerService] })
export class FeatureModule {}
```

- Registers with that module's injector. If the module is eagerly imported, its injector effectively merges into the root environment injector. If lazily loaded, it creates a **new child environment injector**, giving the lazy module its own instance of the service — this is the classic "why do I have two instances of my singleton" trap.

Route-level (`Route.providers` , Angular 14.2+ standalone routing)

```
export const routes: Routes = [
  { path: 'admin', providers: [AdminService], loadComponent: () =>
import('./admin.component') }
];
```

- Creates a new environment injector scoped to that route (and its children), without needing an `NgModule` at all.

Element scope (`@Component / @Directive providers`)

```
@Component({ selector: 'app-card', providers: [CardStatesService] })
export class CardComponent {}
```

- Creates a **new element injector** for every instance of `CardComponent`. Each `<app-card>` in the DOM gets its own `CardStatesService` instance. Destroyed when the component is destroyed.

`viewProviders`

- Like `providers`, but the service is only visible to the component's **own view** (its template), *not* to content projected via `<ng-content>`. Regular `providers` are visible to both the view and projected content children. This distinction is a favorite "gotcha" interview question.

2.4 Provider Recipes

All providers are ultimately objects of the shape `{ provide: Token, ...recipe }` registered in an array. Angular offers shorthand and four explicit recipes.

Shorthand class provider

```
providers: [LoggerService]
// sugar for:
providers: [{ provide: LoggerService, useClass: LoggerService }]
```

`useClass` — provide a different implementation for a token:

```
providers: [{ provide: LoggerService, useClass: ConsoleLoggerService }]
```

Every `injector.get(LoggerService)` returns a `ConsoleLoggerService` instance. Classic for swapping implementations (e.g., mock vs. real service in tests).

`useValue` — provide a static, pre-built value (config objects, constants, mocks):

```
export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG');
providers: [{ provide: APP_CONFIG, useValue: { apiUrl: 'https://api.example.com' } }]
```

No constructor is called — the exact object is returned. Ideal for plain data, feature flags, or fully-mocked services in tests ({ provide: LoggerService, useValue: fakeLogger }).

`useFactory` — compute the instance at runtime, optionally injecting other dependencies via `deps`:

```
export function loggerFactory(config: AppConfig) {
  return config.production ? new ConsoleLoggerService() : new VerboseLoggerService();
}
providers: [
  { provide: LoggerService, useFactory: loggerFactory, deps: [APP_CONFIG] }
]
```

Useful when the choice of implementation depends on runtime configuration, environment, or feature flags.

`useExisting` — alias one token to another *already-registered* provider, without creating a second instance:

```
providers: [
  NewLoggerService,
  { provide: OldLoggerService, useExisting: NewLoggerService }
]
```

Both `OldLoggerService` and `NewLoggerService` tokens resolve to the **same single instance**. This differs from `useClass`, which would instantiate a *second, independent* object.

2.5 Injection Tokens

TypeScript interfaces/types don't exist at runtime — they're erased during compilation — so you cannot use an interface as a DI token. `InjectionToken<T>` is Angular's answer: a unique, generic-typed runtime object usable as a provider key.

```
export interface AppConfig { apiUrl: string; retries: number; }
export const APP_CONFIG = new InjectionToken<AppConfig>('app.config');

// registration
providers: [{ provide: APP_CONFIG, useValue: { apiUrl: '/api', retries: 3 } }]

// consumption
constructor(@Inject(APP_CONFIG) private config: AppConfig) {}
// or, modern:
private config = inject(APP_CONFIG);
```

`InjectionToken` also supports a built-in `providedIn` factory, giving tree-shakable tokens without a separate module registration:

```
export const APP_CONFIG = new InjectionToken<AppConfig>('app.config', {
  providedIn: 'root',
  factory: () => ({ apiUrl: '/api', retries: 3 }),
});
```

2.6 Multi Providers

Normally, registering the same token twice means "last one wins" — the later provider overwrites the earlier one in that injector. Setting `multi: true` instead **accumulates** all providers for that token into an array, which is exactly how Angular implements extensible plugin points like `HTTP_INTERCEPTORS`, `NG_VALIDATORS`, and `APP_INITIALIZER`.

```
export const PLUGIN = new InjectionToken<Plugin>('PLUGIN');

providers: [
  { provide: PLUGIN, useClass: LoggingPlugin, multi: true },
  { provide: PLUGIN, useClass: AnalyticsPlugin, multi: true },
]

// consumer
constructor(@Inject(PLUGIN) private plugins: Plugin[]) {}
// injecting PLUGIN yields [LoggingPluginInstance, AnalyticsPluginInstance]
```

Order is: parent-registered multi entries come before child-registered ones for the same token, if both an ancestor and the current injector register into it (each injector's own accumulated array is appended, walking from root down).

2.7 Resolution Modifier Decorators

These decorators change **where** the injector search starts, and what happens if nothing is found. They only make sense on constructor-parameter injection (or as options object to `inject()`).

`@Optional()` — if no provider is found anywhere up the chain, resolve to `null` instead of throwing `NullInjectorError`.

```
constructor(@Optional() private analytics: AnalyticsService | null) {}
```

`@Self()` — only look in the **requesting component's own element injector**. Do not walk up at all. Throws if not found there (unless combined with `@Optional()`).

```
constructor(@Self() private state: LocalStatesService) {}
```

Used when a directive must consume a service explicitly re-provided at the same host element, and must never accidentally pick up an ancestor's instance.

`@SkipSelf()` — the opposite: skip the current element injector, start looking at the **parent**. Classic use case: a `ControlContainer` or a counter service that must always be the *parent's* instance, never a locally-shadowed one.

```
constructor(@SkipSelf() private parentForm: ControlContainer) {}
```

`@Host()` — walk up element injectors only as far as the **current component's host element**, then stop (don't continue into ancestor components' injectors). Primarily relevant for directives interacting with a host component's providers, and content-projection boundaries. It stops the search at the boundary of the "logical host view."

These can be combined: `@Optional()` `@SkipSelf()` is an extremely common pattern for singleton-guard patterns (see Edge Cases below).

2.8 The `inject()` Function

Introduced as the modern alternative to constructor injection, `inject()` lets you retrieve a dependency **without a constructor parameter**, as long as you're inside an "injection context" (a constructor, a field initializer, a factory function passed to `useFactory`, or explicitly inside `runInInjectionContext()`).

```
@Component({ selector: 'app-widget', standalone: true })
export class WidgetComponent {
  private http = inject(HttpClient);
  private config = inject(APP_CONFIG);
  private analytics = inject(AnalyticsService, { optional: true });
}
```

Signature: `inject<T>(token: ProviderToken<T>, options?: InjectOptions): T`. The options object mirrors the decorators:

```
inject(LoggerService, { optional: true }); // like @Optional()
inject(LoggerService, { self: true });    // like @Self()
inject(LoggerService, { skipSelf: true }); // like @SkipSelf()
inject(LoggerService, { host: true });    // like @Host()
```

Advantages over constructor injection:

- Works in plain functions (e.g., route guards written as functions: `canActivate: () => inject(AuthService).isLoggedIn()`), not just classes.
- Enables mixins/composition — you can extract injection logic into a reusable function without needing inheritance.
- Cleaner with many dependencies — no giant constructor parameter list.
- Required for functional interceptors, functional guards/resolvers (Angular 14.2+ / 15+ router APIs).

Constraint: `inject()` must run synchronously during the "construction phase" — you cannot call it later inside a `setTimeout` or after an `await`, because by then the injection context has been torn down. Angular throws `NG0203: inject() must be called from an injection context` if misused.

3. Code Examples

3.1 Full provider recipe showcase

```
import { InjectionToken, Injectable, inject } from '@angular/core';

export interface AppConfig {
  apiUrl: string;
  featureFlags: Record<string, boolean>;
}

export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG');

@Injectable({ providedIn: 'root' })
export class LoggerService {
  log(msg: string) { console.log(`[LOG] ${msg}`); }
}

export class ConsoleLoggerService extends LoggerService {
  override log(msg: string) { console.log(`[CONSOLE] ${msg}`); }
}

export function loggerFactory(config: AppConfig): LoggerService {
  return config.featureFlags['verboseLogging']
    ? new ConsoleLoggerService()
    : new LoggerService();
}

// bootstrap (standalone)
bootstrapApplication(AppComponent, {
  providers: [
    { provide: APP_CONFIG, useValue: { apiUrl: '/api', featureFlags: { verboseLogging: true } } },
    { provide: LoggerService, useFactory: loggerFactory, deps: [APP_CONFIG] },
  ],
});
```

3.2 useExisting alias vs useClass duplication

```
@Injectable()
export class NewNotificationService {
  notify(msg: string) { console.log('notify:', msg); }
}

// Legacy code still injects `LegacyNotificationService` – alias it, don't duplicate:
@NgModule({
  providers: [
    NewNotificationService,
    { provide: LegacyNotificationService, useExisting: NewNotificationService }, // same instance
  ],
});
```

```

    // { provide: LegacyNotificationService, useClass: NewNotificationService } // WRONG:
    2nd instance
  ],
})
export class SharedModule {}

```

3.3 Multi providers for interceptor-style extension points

```

export const VALIDATION_RULE = new InjectionToken<ValidationRule>('VALIDATION_RULE');

export interface ValidationRule { validate(value: string): string | null; }

@Injectable()
export class RequiredRule implements ValidationRule {
  validate(v: string) { return v ? null : 'Required'; }
}

@Injectable()
export class MaxLengthRule implements ValidationRule {
  validate(v: string) { return v.length <= 20 ? null : 'Too long'; }
}

@Component({
  selector: 'app-field',
  standalone: true,
  providers: [
    { provide: VALIDATION_RULE, useClass: RequiredRule, multi: true },
    { provide: VALIDATION_RULE, useClass: MaxLengthRule, multi: true },
  ],
})
export class FieldComponent {
  private rules = inject(VALIDATION_RULE); // ValidationRule[]

  validateAll(value: string): string[] {
    return this.rules.map(r => r.validate(value)).filter((e): e is string => !!e);
  }
}

```

3.4 @Optional, @Self, @SkipSelf, @Host together

```

@Injectable()
export class TreeNodeCounter {
  count = 0;
}

@Directive({ selector: '[appNode]', providers: [TreeNodeCounter] })
export class NodeDirective {
  constructor(
    @Self() private ownCounter: TreeNodeCounter,
    @Optional() @SkipSelf() private parentCounter: TreeNodeCounter | null,
    @Optional() private ambient: AnalyticsService | null,
  ) {}
}

```



```

    ) {
      if (this.parentCounter) {
        // roll counts up to the nearest ancestor that also has TreeNodeCounter
        this.parentCounter.count += 1;
      }
    }
  }
}

```

3.5 inject() in a functional guard (no class needed)

```

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.isLoggedIn() ? true : router.createUrlTree(['/login']);
};

export const routes: Routes = [
  { path: 'admin', canActivate: [authGuard], loadComponent: () =>
import('./admin.component') },
];

```

3.6 inject() inside a reusable composition function

```

function useDestroyRef() {
  const destroyRef = inject(DestroyRef);
  return destroyRef;
}

@Component({ selector: 'app-timer', standalone: true })
export class TimerComponent {
  private destroyRef = useDestroyRef(); // still valid: called during construction
  constructor() {
    const id = setInterval(() => console.log('tick'), 1000);
    this.destroyRef.onDestroy(() => clearInterval(id));
  }
}

```

4. Internal Working

4.1 Building the environment injector tree

At bootstrap, Angular constructs a chain:

```

NullInjector → Platform Injector → Root Environment Injector → Route/Lazy Environment
Injectors (per lazy chunk)

```

- The **Platform injector** is created once per browser context by `platformBrowser()` / `platformBrowserDynamic()`. It hosts `providedIn: 'platform'` services.

- The **Root environment injector** is created by `bootstrapApplication()` (standalone) or by bootstrapping the root `NgModule`. It merges: the app's own `providers` array, all eagerly-loaded `NgModules`' `providers`, and every `providedIn: 'root'` `@Injectable` (registered lazily as a "record," not yet instantiated).
- Each **lazy-loaded route** (via `loadChildren` / `loadComponent` with its own `providers`) creates a **new child environment injector**, whose parent is the root environment injector (or a nested parent route injector). This is why lazy features can have their own instance of a `providedIn: 'root'`-scoped-but-locally-overridden service, or their own route-scoped providers entirely.

Environment injectors are built from "definitions" — Angular walks the `providers` arrays (and the `export/import` graph for `NgModules`) at configuration time and flattens them into a single provider record map per injector, resolving `multi` accumulation as it goes.

4.2 Building the element injector tree

Every component/directive gets an entry in Ivy's `LView`/`TView` data structures. If a component (or any directive on the same host element) declares `providers` or `viewProviders`, Ivy allocates space in that element's `LView` for those provider records — this is often called the **Node Injector**. If a component/directive has *no* providers, Angular does **not** allocate a separate injector node — it's skipped entirely during the walk for performance, effectively making the search "logically" pass through it without cost.

The element injector chain mirrors the **component tree** (not the raw DOM tree): `ChildComponent's injector` → `ParentComponent's injector` → ... → `RootComponent's injector`. When the element-injector chain is exhausted (i.e., we reach the injector for the root component and it doesn't have what we need, or there is no relevant provider anywhere in the component tree), Angular falls through to the **environment injector chain** at the top (route injector → root environment injector → platform injector → `NullInjector`).

4.3 The resolution algorithm (walk-up order)

For a dependency requested inside component `C`:

1. Start at `C`'s own element injector (if it has providers). Check `@Self()` constraint here if present.
2. If not `@Self()`-restricted and not found, walk up through ancestor **element injectors** (parent component, grandparent component, ...), honoring `@Host()` (stop at the host boundary) and `@skipSelf()` (skip step 1 entirely, start at step 2).
3. If still not found after exhausting element injectors, move into the **environment injector chain**: the closest route/module environment injector, then its parent (up to root), then platform.
4. If nothing anywhere provides the token, delegate to the `NullInjector`.
5. `NullInjector.get(token)` always throws `NullInjectorError: No provider for x!` — **unless** the request had `@Optional()`, in which case the *caller* (not the `NullInjector` itself) catches this and substitutes `null`.

Each injector, once it resolves a class/factory provider, **caches** the resulting instance keyed by token, so subsequent requests to that same injector return the cached singleton instantly without re-running the factory/constructor.

4.4 Why `providedIn: 'root'` is tree-shakable

With the older `NgModule.providers: [LoggerService]` style, the class name has to appear literally in a `providers` array in some file the compiler processes — the reference keeps it alive even if unused, and duplicate NgModules that each list it can cause duplicate registration confusion.

With `@Injectable({ providedIn: 'root' })`, Angular's compiler embeds the provider recipe **directly on the class itself** (`@prov` static property) rather than in an external array. The root injector doesn't need a manually maintained list — it just checks, at injection time, "does this token have a `@prov` with `providedIn: 'root'`?" and lazily registers/instantiates it on first use. If the class is never imported/injected anywhere in the compiled output, nothing references it, and standard JS dead-code elimination (terser) can drop it from the final bundle — true tree-shaking. This is why Angular's docs call it "tree-shakable providers."

4.5 The `NullInjector`

The `NullInjector` sits conceptually above the platform injector, as the final link in every chain. Its `get()` implementation is intentionally trivial:

```
class NullInjector implements Injector {
  get(token: any, notFoundValue?: any): any {
    if (notFoundValue !== NOT_FOUND_VALUE_MARKER) return notFoundValue;
    throw new Error(`NullInjectorError: No provider for ${stringify(token)}!`);
  }
}
```

`@Optional()` works by having the resolution machinery pass a special "not found" sentinel down to the `NullInjector` call and translating that sentinel into `null` at the call site, rather than the `NullInjector` "knowing" about `@Optional()` itself.

5. Edge Cases & Gotchas

"Singleton" service that isn't singleton. The #1 DI trap: a service is `@Injectable({ providedIn: 'root' })`, so developers assume "one instance app-wide." But if *any* component's `providers` array also lists that same class, every instance of that component gets its **own private instance**, shadowing the root one for that component's subtree. This is not a bug — it's the element injector correctly taking priority over the environment injector — but it silently breaks shared state (e.g., a shared cart/counter service) if someone "helpfully" adds it to a component's `providers` for testing and forgets to remove it.

Lazy-loaded modules duplicating services. In the `NgModule` era, if a lazy-loaded feature module lists `providers: [SomeRootService]` (even redundantly), it creates a second instance scoped to that lazy module's injector, independent of the root instance. The fix is simply: don't re-list `providedIn: 'root'` services in feature module providers unless you deliberately want an isolated instance.

`viewProviders` invisible to projected content. A service in `viewProviders` is available to the component's template bindings and child components declared in that template, but **not** to components/directives passed in via `<ng-content>` — those retrieve their dependencies from *their own* originating template's injector chain, not the component they're projected into. Interview trap: "why can't my projected child see the parent's `viewProvider` service?" — because it was never logically inside that view.

Circular DI errors (NG0200: Circular dependency). If `ServiceA` injects `ServiceB` in its constructor, and `ServiceB` injects `ServiceA`, Angular throws a circular dependency error at construction time, because building A requires B to already exist, which requires A to already exist. Fixes: refactor to remove the cycle (extract shared state into a third service), use `Injector.get()` lazily inside a method instead of the constructor, or (for forward-referenced *classes*, not real cycles) use `forwardRef()`.

`forwardRef()` for not-yet-defined classes. This solves a *different* problem than true circular DI: when a class needs to reference another class as a provider token *before that class has been declared* (common with two decorated classes in the same file referencing each other, or self-referential providers):

```
@Component({
  selector: 'app-item',
  providers: [{ provide: PARENT_TOKEN, useExisting: forwardRef(() => ItemComponent) }],
})
export class ItemComponent {}
```

`forwardRef()` wraps a lambda that's only evaluated when Angular actually needs the value, deferring the reference until after all classes in the module have been defined — it's a JS temporal-dead-zone workaround, not a DI-architecture fix.

`@Host()` boundary confusion. Developers often expect `@Host()` to mean "only the direct parent," but it actually means "walk up element injectors until you hit the boundary of the current component's host view" (which can include several directive layers on the same host element, and stops before crossing into an ancestor component that projected this one in). It's subtly different from `@self()` (only this exact element) and different from unrestricted walking.

`@Optional()` masking real bugs. Overusing `@Optional()` "just in case" can turn a genuine misconfiguration (forgot to provide the service) into a silent `null` that causes a `TypeError` much later, far from the real root cause, making debugging harder. Use it only when the collaborator is genuinely optional by design.

Constructor injection order + property initializer timing. Angular resolves constructor parameters before any field initializers/decorators run, but Angular decorators like `@Input()` are not yet populated inside the constructor — a common confusion is expecting `@Input()` values to be available in DI-resolved constructor logic; they're only guaranteed inside `ngOnInit()`.

`inject()` outside an injection context. Calling `inject()` inside a `setTimeout`, a `Promise.then()`, or after an `await` throws `NG0203`. The fix is to capture the injected value synchronously first, or wrap the deferred logic with `runInInjectionContext(injector, () => ...)`.

Multi providers and ordering assumptions. Code that iterates a `multi: true` array assuming a specific registration order can break if a lazy-loaded child injector contributes its own entries — child-injector multi entries are appended after the parent's for a request resolved at the child level. Don't rely on positional array order for correctness when providers may exist at multiple injector levels; validate/sort explicitly if order matters.

6. Interview Questions & Answers

Q1. What is Dependency Injection, and why does Angular use it instead of classes just instantiating what they need?

DI is a pattern where a class declares its dependencies (usually via constructor parameters or `inject()`) rather than constructing them itself. Angular's injector resolves and supplies those dependencies at runtime. This decouples classes from concrete implementations, makes unit testing trivial (swap in mocks via `useValue` / `useClass` in `TestBed`), enables tree-shakable singletons, and lets Angular manage object lifetimes (one instance per app, per route, or per component) transparently.

Q2. What's the difference between `useClass`, `useValue`, `useFactory`, and `useExisting`?

`useClass` instantiates the given class (`new`) whenever that token is requested — good for swapping implementations. `useValue` returns a pre-built, static object with no instantiation — good for constants/config/mocks. `useFactory` calls a function (optionally injecting its own `deps`) to compute the value at runtime — good when the choice of implementation depends on other providers or runtime conditions. `useExisting` aliases a token to another **already registered** token so both resolve to the exact same single instance — useful for renaming/deprecating a token without creating a duplicate object (unlike `useClass`, which would create a second instance).

Interviewer intent: This question checks whether the candidate actually understands *object identity* differences (`useExisting` vs `useClass`), not just that "there are four provider types" — a very common surface-level gap.

Q3. Explain `providedIn: 'root'` vs registering a service in an `NgModule`'s `providers` array.

Both eventually make the service available application-wide if eagerly loaded, but `providedIn: 'root'` embeds the provider recipe on the class itself (`@prov`), letting Angular's build tooling tree-shake the service out entirely if it's never injected anywhere — true dead-code elimination. `NgModule.providers` requires the class to be referenced in an array that the compiler processes, so it can't be shaken the same way, and if that module is lazy-loaded, it creates a *separate* instance scoped to that lazy injector rather than a shared root instance — a classic source of "duplicate singleton" bugs.

Q4. A service is `@Injectable({ providedIn: 'root' })`, but two sibling components each show different data from it. What's the likely cause?

Almost certainly one (or both) of those components — or an ancestor between them and the root — lists that same service class in its own `providers` array, which creates a new element-injector-scoped instance for that subtree, shadowing the root singleton. Because element injectors are checked before falling through to the environment injector, the shadowed instance wins for anything inside that component's tree. The fix is to remove the redundant `providers` entry so all components resolve to the single root instance.

Interviewer intent: Tests whether the candidate can diagnose a very common real-world bug from symptoms alone, showing they understand injector *hierarchy precedence*, not just definitions.

Q5. What does `@Optional()` do, and what happens without it if no provider exists?

Without `@Optional()`, if no injector in the entire chain (element injectors up to the environment injectors up to platform) has a provider for the requested token, resolution reaches the `NullInjector`, which throws `NullInjectorError: No provider for X!`. `@Optional()` catches that outcome and substitutes `null` instead of throwing, letting the class handle a missing dependency gracefully (e.g., an optional third-party analytics service that may not be configured in every environment).

Q6. Explain `@Self()` vs `@SkipSelf()` with an example of when you'd use each.

`@Self()` restricts the search to the injector on the exact same element as the requesting class — it never walks up, and throws (unless also `@Optional()`) if not found there. It's used when a directive must use a service explicitly re-provided at its own host element, guaranteeing it never silently inherits an ancestor's instance. `@SkipSelf()` does the opposite: it deliberately skips the current element's own injector and starts searching at the parent, which is used when a component wants to access/aggregate the *parent's* instance of a service rather than a locally shadowed one — e.g., nested `FormControl`/form-group scenarios, or a tree-node counter that must find its parent's counter, not create/read its own.

Q7. What is `@Host()` and how does it differ from unrestricted upward search?

`@Host()` limits the upward walk through element injectors to the boundary of the requesting component's host view — it will traverse sibling directives on the same host element and stop there, rather than continuing into ancestor components' injectors. It's typically used inside directives that need to interact only with providers local to the component they're attached to, respecting the content-projection boundary, without accidentally reaching into some unrelated ancestor component several levels up.

Q8. What is an `InjectionToken` and why can't you just use a TypeScript interface as a DI token?

Interfaces and type aliases are a compile-time-only construct — they're erased by the TypeScript compiler and don't exist at runtime, so there's nothing for Angular's injector to use as a map key. `InjectionToken<T>` creates a unique runtime object (essentially a typed symbol) that can be used as a `provide` key while still giving you compile-time type safety (`T`) when you inject it. It's also used for any non-class dependency — config objects, primitive values, arrays of multi-providers — where there's no class constructor to use as the token.

Q9. What are multi providers, and give a real Angular API that uses them.

Multi providers (`multi: true`) let multiple registrations for the *same token* accumulate into an array instead of the last one overwriting the others. Angular's `HTTP_INTERCEPTORS` token is the canonical example: every interceptor you register with `multi: true` gets added to the array Angular iterates through when building the interceptor chain for outgoing HTTP requests, letting features/modules each contribute their own interceptor without needing to know about each other.

Q10. Walk through Angular's exact resolution algorithm when a component requests a dependency.

Angular first checks the requesting component's own element injector (if it has one), respecting `@Self()`. If not restricted to self and not found, it walks up through ancestor element injectors (parent, grandparent components with providers), respecting `@Host()`'s stop boundary and `@SkipSelf()`'s "skip my own level" instruction. If the element injector chain is exhausted without a match, resolution moves into the environment injector chain — starting at the nearest route/lazy-module environment injector, up through the root environment injector, up to the platform injector. If still nothing matches, it falls to the `NullInjector`, which throws unless `@Optional()` catches the miss and substitutes `null`. Each injector along the way caches any instance it resolves, so repeated requests at that level are $O(1)$ afterward.

Interviewer intent: This is the load-bearing "do you actually understand the internals" question. A strong answer names both hierarchies explicitly and gets the order right (element injectors first, then environment injectors) rather than treating it as one flat chain.

Q11. What's the difference between the environment injector hierarchy and the element injector hierarchy?

The environment injector hierarchy (platform → root → route/lazy-module injectors) is built from `providedIn`, `NgModule.providers`, `bootstrapApplication({providers})`, and `Route.providers` — it's long-lived and mirrors the app/route structure, not the DOM. The element injector hierarchy is built per component/directive

instance whenever `@Component / @Directive` declares `providers` or `viewProviders` — it mirrors the live component tree and is destroyed/recreated alongside those component instances. Resolution always checks the element injector hierarchy first (nearest component outward) before falling through to the environment injector hierarchy at the top, which is why a component-level provider can override a root-level "singleton."

Q12. What is the `inject()` function, where can it be used, and what are its constraints compared to constructor injection?

`inject()` retrieves a dependency imperatively from within an "injection context" — a constructor, a field initializer, a factory function, or code wrapped in `runInInjectionContext()` — without requiring a constructor parameter. Unlike constructor injection, it works inside plain functions, which is what makes Angular's functional router guards/resolvers/interceptors possible (`canActivate: () => inject(AuthService).isLoggedIn()`), and it enables extracting reusable "composition functions" that call `inject()` internally without needing base classes or decorators. Its key constraint: it must execute synchronously during construction — calling it after an `await`, inside a `setTimeout`, or in an event callback throws `NG0203: inject() must be called from an injection context`, because the ambient injection context that `inject()` reads from only exists during that synchronous construction window.

Q13. How does `forwardRef()` differ from a genuine circular dependency error, and when would you use it?

A genuine circular DI error (`NG0200`) happens when constructing service A requires service B to already exist, and constructing B requires A — an unresolvable runtime cycle that must be fixed by refactoring (extract shared state, use lazy `Injector.get()` inside a method instead of the constructor). `forwardRef()` addresses a narrower, purely syntactic problem: referencing a class as a provider token before it has been declared in the file (e.g., two decorated classes referencing each other, or a component providing itself under another token). `forwardRef(() => SomeClass)` wraps the reference in a lambda that Angular evaluates lazily, after all classes are defined, sidestepping the JS temporal-dead-zone issue — it doesn't fix an actual instantiation cycle, only a declaration-order problem.

Q14. Why might a service marked `providedIn: 'root'` still end up with multiple instances in a lazy-loaded-module application?

If a lazy-loaded feature module (or a lazy route's `providers` array) explicitly re-lists that service, Angular creates a new child environment injector for that lazy chunk, and that child's own provider record takes precedence for anything resolved within that lazy chunk's subtree — giving it a separate instance from the one held by the root environment injector. This is also intentional behavior for `providedIn: 'any'`, which is *designed* to give each lazy module its own instance. The fix, when unintentional, is to remove the redundant registration and let the lazy module inherit the root instance via normal injector delegation.

7. Quick Revision Cheat Sheet

Provide scopes

Scope	Where registered	Instance lifetime
<code>providedIn: 'root'</code>	Root environment injector	One per app (tree-shakable)
<code>providedIn: 'platform'</code>	Platform injector	Shared across all apps in the page
<code>providedIn: 'any'</code>	Root or per lazy module	Root: shared; lazy: one per lazy module

Scope	Where registered	Instance lifetime
<code>NgModule.providers</code>	That module's injector	Eager: merges into root; lazy: new instance
<code>Route.providers</code>	Route's environment injector	One per route load
<code>@Component/@Directive providers</code>	Element injector	One per component/directive instance
<code>@Component viewProviders</code>	Element injector, view-only	One per instance, invisible to projected content

Provider recipes

- `useClass` → instantiate a class.
- `useValue` → return a static value, no instantiation.
- `useFactory` (+ `deps`) → compute value at runtime.
- `useExisting` → alias to another token's *same* instance (no duplication).
- `multi: true` → accumulate into an array instead of overwrite.

Resolution modifiers

- `@Optional()` → resolve to `null` instead of throwing if not found.
- `@Self()` → only check the requesting element's own injector.
- `@SkipSelf()` → skip own injector, start at parent.
- `@Host()` → walk up only to the host view boundary.
- Combine freely: `@Optional() @skipSelf()` is a common singleton-guard pattern.

Resolution order: requesting element injector → ancestor element injectors (respecting modifiers) → nearest environment injector (route/lazy) → root environment injector → platform injector → `NullInjector` (throws, or `null` if `@Optional()`).

`inject()`

- Same resolution rules, callable in functions/factories, not just constructors.
- Must run synchronously in an injection context; use `runInInjectionContext()` for deferred use.
- Options: `{ optional, self, skipSelf, host }` mirror the decorators.

Tree-shaking: `providedIn: 'root'` puts the recipe on the class (`@prov`) so unused services can be dropped by dead-code elimination; `NgModule.providers` arrays can't be shaken the same way.

Top gotchas to remember: component-level `providers` shadow root singletons; lazy modules re-listing a root service creates a second instance; `viewProviders` is invisible to `<ng-content>`; `forwardRef()` fixes declaration order, not real circular dependency errors; `inject()` fails outside a synchronous injection context (NG0203).

